CodePulse

API Documentation

MERN Stack DSA Interview Prep & Progress Tracking Platform

Base URL	http://localhost:5000/api
Authentication	JWT Bearer Token
Total Endpoints	30
Modules	Auth, Topics, Problems, Dashboard, Goals, Profile, Support, AI Study Plan
Author	Varinda Aggarwal
Last Verified	August 2026 — every endpoint tested via Postman against live backend

API Summary

A quick reference before the detailed endpoint-by-endpoint documentation below.

Module	Method Types	Purpose
Authentication	POST, GET	Register (OTP-based), login, Google OAuth, password reset
User / Profile	GET, PUT, DELETE	View/edit profile, manage profile photo
Topics	GET, POST, PUT, DELETE	Manage DSA topics being tracked
Problems	GET, POST, PUT, DELETE	Log and track individual solved problems
Dashboard	GET	Aggregated progress analytics and mastery scores
Daily Goal	GET, POST	Set and track daily problem-solving targets
AI Study Plan	GET, POST, PATCH	Generate and track AI-personalized study plans
Help & Support	POST	Submit contact messages, bug reports, feature requests

Overview

CodePulse is a full-stack MERN application for tracking Data Structures & Algorithms interview preparation. The backend exposes a REST API built with Express and MongoDB (Mongoose), covering authentication, topic and problem tracking, dashboard analytics, daily goals, profile management, support tickets, and AI-generated study plans.

Base URL

Development	`http://localhost:5000/api`
Production	`https://codepulse-646c.onrender.com/api`

Common Headers

Content-Type	`application/json` `(default for all endpoints)`
Content-Type	`multipart/form-data` `(only for PUT /profile, when uploading a photo)`
Authorization	`Bearer <token>` `(only required on endpoints marked "Required" below)`

Authentication

Most endpoints require a JWT, obtained from Login or Verify OTP. Send it as a Bearer token:

```
Authorization: Bearer <token>
```

Tokens are valid for 30 days. Auth routes (including Google OAuth start/callback) are the only public endpoints — everything else is marked "Required" in its endpoint header below and needs a valid token, or returns 401.

Error Response Format

API errors return an HTTP status code along with a JSON body containing a descriptive message field. Validation errors (from express-validator) return an errors array of field-level issues instead. Examples of both, matching what this backend actually returns:

```
General error:
{
  "message": "Invalid credentials"
}

Validation error:
{
  "errors": [
    { "msg": "Please enter a valid email", "path": "email", ... }
  ]
}
```

Signup Flow

Signup is a two-step OTP flow, not a single /register call: POST /auth/send-otp (sends a 6-digit code, valid 5 minutes) followed by POST /auth/verify-otp (creates the account and returns a token). Google OAuth is a one-step alternative.

Rate Limiting

Login	5 requests / 15 min
Send OTP / Verify OTP	5 requests / 15 min (shared)
Forgot / Reset Password	3 requests / 15 min (keyed by IP + email)
AI Study Plan generation	2 generations / day per user

Tech Stack

Node.js, Express, MongoDB with Mongoose, JWT authentication, Passport.js (Google OAuth 2.0), Cloudinary (profile photo storage via Multer), Gmail API (email delivery via OAuth2/HTTPS, not raw SMTP), express-validator (request validation), express-rate-limit (abuse protection), and Google Gemini (gemini-flash-latest) for AI study plan generation.

Response Status Codes

Standard codes used across this API:

Code	Name	Meaning
400	Bad Request	The request failed validation, or violated a business rule (e.g. duplicate name)
401	Unauthorized	Missing/invalid JWT, or the requester doesn't own the resource
403	Forbidden	Reserved for future use — not currently returned by this API
404	Not Found	The requested resource does not exist
409	Conflict	Reserved for future use — this API currently returns 400 for duplicate-key conflicts
500	Internal Server Error	An unexpected server-side error

Additional codes specific to this API's behavior:

Code	Name	Meaning
200	OK	Request succeeded
201	Created	Resource created successfully
302	Found	Redirect (used only by the Google OAuth routes)
429	Too Many Requests	Rate limit exceeded (login/OTP/reset attempts, or the AI daily generation cap
502	Bad Gateway	The AI provider (Gemini) returned a response that couldn't be parsed or validate
503	Service Unavailable	The AI provider is temporarily overloaded; the backend already retried automatically before return

Table of Contents

Authentication APIs

/api/auth

Handles registration (via OTP verification), login, Google OAuth, and password reset.

POST `/api/auth/send-otp` **Public**

Step 1 of signup. Sends a 6-digit OTP to the given email, valid for 5 minutes (auto-expires via MongoDB TTL index). Signup data (username, phone, hashed password) is held temporarily until OTP verification.

Rate limit: 5 requests / 15 min

REQUEST BODY

```
{
  "username": "varinda",
  "email": "varinda@example.com",
  "phone": "9876543210",
  "password": "Passw0rd!"
}
```

password must be 8+ chars with 1 uppercase, 1 lowercase, 1 digit, 1 special character (@$!%*?&#)

SUCCESS RESPONSE 200

```
{
  "message": "OTP sent to your email"
}
```

ERROR RESPONSES

400	Validation error, or user already exists with this email
429	Too many OTP requests

POST /api/auth/verify-otp Public

Step 2 of signup. Verifying the correct OTP creates the User account and returns a JWT. Account is brand-new at this point, so firstName/lastName/photo are not yet set.

Rate limit: 5 requests / 15 min (shared with send-otp)

REQUEST BODY

```
{
  "email": "varinda@example.com",
  "otp": "482913"
}
```

SUCCESS RESPONSE 201

```
{
  "_id": "66b1f2a4c9d2e1a1b2c3d4e5",
  "username": "varinda",
  "email": "varinda@example.com",
  "token": "eyJhbGciOiJIUzI1NiIs... (truncated)"
}
```

ERROR RESPONSES

400	Invalid or expired OTP, or validation error
400	Username already taken (duplicate key, returned as a clean message)
429	Too many requests

POST `/api/auth/login` **Public**

Authenticates an existing user with email + password.

Rate limit: 5 requests / 15 min

REQUEST BODY

```
{
  "email": "varinda@example.com",
  "password": "Passw0rd!"
}
```

SUCCESS RESPONSE 200

```
{
  "_id": "66b1f2a4c9d2e1a1b2c3d4e5",
  "username": "varinda",
  "email": "varinda@example.com",
  "firstName": "Varinda",
  "lastName": "Aggarwal",
  "photo": null,
  "token": "eyJhbGciOiJIUzI1NiIs... (truncated)"
}
```

ERROR RESPONSES

400	Invalid credentials
429	Too many login attempts

POST /api/auth/forgot-password Public

Sends a password reset link (valid 15 minutes) if the email is registered. Always returns the same generic message whether or not the email exists, to prevent account enumeration.

Rate limit: 3 requests / 15 min (keyed by IP + email combo)

REQUEST BODY

```
{
  "email": "varinda@example.com"
}
```

SUCCESS RESPONSE 200

```
{
  "message": "If that email exists, a reset link has been sent"
}
```

ERROR RESPONSES

400	Validation error
429	Too many requests

POST /api/auth/reset-password/:token Public

Completes the password reset using the token from the emailed link.

Rate limit: Shared with forgot-password

REQUEST BODY

```
{
  "password": "NewPassw0rd!",
  "confirmPassword": "NewPassw0rd!"
}
```

SUCCESS RESPONSE 200

```
{
  "message": "Password reset successful. You can now log in."
}
```

ERROR RESPONSES

400	Validation error, or passwords don't match
400	Invalid or expired reset link
404	User not found

GET /api/auth/google Public

Redirects to Google's account picker to begin OAuth login/signup. This is a browser-redirect flow, not a typical JSON endpoint — open it directly in a browser rather than calling it from Postman.

Rate limit: None

SUCCESS RESPONSE 302

```
Redirects to Google's OAuth consent screen
```

GET /api/auth/google/callback Public

Google redirects here after account selection. Passport verifies the profile (creating a new user by googleId/email match, or logging in an existing one — see the passport.js strategy), then this route handler signs a fresh JWT and redirects to {FRONTEND_URL}/auth/success?token=<JWT>. The frontend is expected to read the token query parameter from that URL and store it. New Google-signup users get a random, unusable password placeholder stored in the database — they authenticate via Google only; no set-password endpoint currently exists for these accounts. The redirect URL is read from the FRONTEND_URL environment variable (e.g. the deployed Vercel URL in production, http://localhost:3000 in development) — no longer hardcoded. passport.initialize() is registered before the /api/auth mount in index.js, ensuring it runs before these Google OAuth routes are hit.

Rate limit: None

SUCCESS RESPONSE 302

```
Redirects to frontend with token
```

User / Profile APIs

/api/profile

View and edit the logged-in user's profile, including photo upload/removal via Cloudinary.

GET `/api/profile` **Required**

Returns the full profile: personal fields plus computed stats (totalProblems, totalTopics, completedTopics) and a computed hasPassword flag. hasPassword is derived at request time (true only if a real password is set AND the account isn't Google-linked) — it is not a stored database field. Note: no endpoint currently exists for a Google-only user to set a password.

SUCCESS RESPONSE 200


```
{
  "_id": "66b1f2a4c9d2e1a1b2c3d4e5",
  "username": "varinda",
  "email": "varinda@example.com",
  "firstName": "Varinda",
  "lastName": "Aggarwal",
  "phone": "9876543210",
  "dob": "2004-05-15",
  "country": "India",
  "photo": "https://res.cloudinary.com/codepulse/image/upload/v1/codepulse/profiles/abc123.jpg",
  "joinedDate": "2026-05-20T09:00:00.000Z",
  "totalProblems": 84,
  "totalTopics": 12,
  "completedTopics": 5,
  "hasPassword": true
}
```


ERROR RESPONSES

401	Not authorized

PUT `/api/profile` **Required**

Partial update — only send fields you want to change. Supports optional photo upload (multipart field name 'photo'), stored via Cloudinary and auto-cropped to 300x300.

REQUEST BODY

```
{
  "firstName": "Varinda",
  "lastName": "Aggarwal",
  "phone": "9876543210 (10-digit Indian format)",
  "dob": "2004-05-15 (ISO8601)",
  "country": "India",
  "photo": "(file, multipart/form-data)"
}
```

SUCCESS RESPONSE 200

```
{
  "_id": "66b1f2a4c9d2e1a1b2c3d4e5",
  "username": "varinda",
  "email": "varinda@example.com",
  "firstName": "Varinda",
  "lastName": "Aggarwal",
  "phone": "9876543210",
  "dob": "2004-05-15",
  "country": "India",
  "photo": "https://res.cloudinary.com/codepulse/image/upload/v1/codepulse/profiles/abc123.jpg",
  "hasPassword": true
}
```

Note: there is no message wrapper on this response — it returns the updated profile fields directly (unlike most other write endpoints in this API, which include a message field).

ERROR RESPONSES

400	Validation error
401	Not authorized

DELETE `/api/profile/photo` Required

Removes the current profile photo — deletes it from Cloudinary (extracts the publicId from the stored URL) and clears the photo field.

SUCCESS RESPONSE 200

```
{
  "message": "Photo deleted successfully"
}
```

Note: this endpoint's success message follows a different wording pattern ("X deleted successfully") than Topics/Problems delete ("X removed") — a minor inconsistency in the actual codebase, documented here as-is rather than smoothed over.

ERROR RESPONSES

401	Not authorized

Topics APIs

/api/topics

CRUD for DSA topics the user is tracking (e.g. Arrays, Dynamic Programming).

GET `/api/topics` **Required**

Returns the logged-in user's topics, with optional search and sort.

QUERY PARAMETERS

`search`	(optional, matches name)
`sort`	latest | oldest | name | revision

SUCCESS RESPONSE 200


```
[
  {
    "_id": "66b1f2a4c9d2e1a1b2c3d4e6",
    "name": "Dynamic Programming",
    "status": "In Progress",
    "completedAt": null,
    "needsRevision": false,
    "createdAt":  "2026-01-10T09:12:00.000Z"
  },
  {
    "_id": "66b1f2a4c9d2e1a1b2c3d4e7",
    "name": "Graphs",
    "status": "Not Started",
    "completedAt": null,
    "needsRevision": false,
    "createdAt":  "2026-01-12T14:05:00.000Z"
  }
]
```


ERROR RESPONSES

401	Not authorized

POST `/api/topics` **Required**

Creates a new topic. Topic name must be unique per-user (case-insensitive check).

REQUEST BODY

```
{
  "name": "Dynamic Programming",
  "status": "Not Started"
}
```

SUCCESS RESPONSE 201

```
{
  "_id": "66b1f2a4c9d2e1a1b2c3d4e6",
  "user": "66b1f2a4c9d2e1a1b2c3d4e5",
  "name": "Dynamic Programming",
  "status": "Not Started",
  "completedAt": null,
  "needsRevision": false
}
```

ERROR RESPONSES

400	Topic "<name>" already exists, or validation error
401	Not authorized

PUT `/api/topics/:id` **Required**

Updates a topic's name and/or status. Setting status to 'Done' auto-sets completedAt; moving away from 'Done' clears it. Ownership is checked. The full request body is forwarded directly to the database update via findByIdAndUpdate (no field whitelist), so needsRevision is also settable here even though it isn't accepted on creation (POST only reads name and status). Note: unlike POST, this route has no validateTopic middleware, and findByIdAndUpdate is called without runValidators — so an invalid status value (outside the enum) is NOT rejected here; it would be saved as-is, bypassing the schema's enum constraint entirely.

REQUEST BODY

```
{
  "name": "Dynamic Programming",
  "status": "Done",
  "needsRevision": true
}
```

SUCCESS RESPONSE 200

```
{
  "_id": "66b1f2a4c9d2e1a1b2c3d4e6",
  "name": "Dynamic Programming",
  "status": "Done",
  "completedAt": "2026-01-15T18:20:00.000Z",
  "needsRevision": true
}
```

ERROR RESPONSES

400	Topic "<name>" already exists (on rename)
401	Not authorized (ownership check — same message text as a missing/invalid token, per the controller)
404	Topic not found

DELETE `/api/topics/:id` Required

Deletes a topic. Ownership is checked.

SUCCESS RESPONSE 200

```
{
  "message": "Topic removed"
}
```

ERROR RESPONSES

401	Not authorized (ownership check — same message text as a missing/invalid token, per the controller)
404	Topic not found

Problems APIs

/api/problems

CRUD for individual DSA problems logged under a topic.

GET `/api/problems` **Required**

Returns the logged-in user's problems with search, filtering, sorting, and pagination.

QUERY PARAMETERS

`search`	(optional)
`difficulty`	Easy | Medium | Hard (optional)
`topic`	(optional, topic id)
`sort`	newest | oldest (optional)
`page`	optional, default: 1
`limit`	optional, default: 10

topic in each result is populated with the topic's _id and name (not a raw ObjectId), via .populate('topic', 'name').

SUCCESS RESPONSE 200


```
{
  "problems": [
    {
      "_id": "66b1f2a4c9d2e1a1b2c3d4f1",
      "name": "Longest Increasing Subsequence",
      "difficulty": "Medium",
      "status": "Solved",
      "topic": {
        "_id": "66b1f2a4c9d2e1a1b2c3d4e6",
        "name": "Dynamic Programming"
      },
      "dateSolved": "2026-01-14T11:00:00.000Z"
    },
    {
      "_id": "66b1f2a4c9d2e1a1b2c3d4f2",
      "name": "Course Schedule",
      "difficulty": "Medium",
      "status": "In Progress",
      "topic": {
        "_id": "66b1f2a4c9d2e1a1b2c3d4e7",
        "name": "Graphs"
      },
      "dateSolved": null
    }
  ],
  "currentPage": 1,
  "totalPages": 5,
  "totalProblems": 42
}
```

ERROR RESPONSES

401	Not authorized

POST `/api/problems` Required

Creates a new problem under a topic. Duplicate check is by name OR link. Setting status to 'Solved' (the default if omitted) auto-sets dateSolved to the current time. Note: approach, timeComplexity, spaceComplexity, and needsRevision exist on the Problem model but are NOT accepted at creation time — the controller only reads name, link, difficulty, topic, notes, and status from the request body. Use PUT to add those fields afterward (see below).

REQUEST BODY

```
{
  "name": "Longest Increasing Subsequence",
  "link": "https://leetcode.com/problems/longest-increasing-subsequence/",
  "difficulty": "Medium",
  "status": "Solved",
  "topic": "66b1f2a4c9d2e1a1b2c3d4e6",
  "notes": "Classic DP problem"
}
```

SUCCESS RESPONSE 201

```
{
  "_id": "66b1f2a4c9d2e1a1b2c3d4f1",
  "name": "Longest Increasing Subsequence",
  "difficulty": "Medium",
  "status": "Solved",
  "topic": "66b1f2a4c9d2e1a1b2c3d4e6",
  "notes": "Classic DP problem",
  "approach": null,
  "timeComplexity": null,
  "spaceComplexity": null,
  "needsRevision": false,
  "dateSolved":  "2026-01-15T10:30:00.000Z"
}
```

ERROR RESPONSES

400	A problem named "<name>" already exists, or A problem with this link already exists, or validation error
401	Not authorized

PUT `/api/problems/:id` **Required**

Updates a problem. Ownership checked. Duplicate check re-runs if name or link is changed. dateSolved is auto-managed on status transitions. The full request body is forwarded directly to the database update via find By Id And Update (no field whitelist), so this is also the only way to set approach, timeComplexity, spaceComplexity, and needsRevision — they are not accepted on creation (see POST above). Note: unlike POST, this route has no validateProblem middleware, and findByIdAndUpdate is called without runValidators — so an invalid difficulty value (outside the enum) is NOT rejected here; it would be saved as-is.

REQUEST BODY

```
{
  "status": "Solved",
  "notes": "Revisited and solved cleanly",
  "approach": "O(n^2) DP with binary search optimization to O(n log n)",
  "timeComplexity": "O(n log n)",
  "spaceComplexity": "O(n)",
  "needsRevision": true
}
```

SUCCESS RESPONSE 200

```
{
  "_id": "66b1f2a4c9d2e1a1b2c3d4f1",
  "status": "Solved",
  "notes": "Revisited and solved cleanly",
  "approach": "O(n^2) DP with binary search optimization to O(n log n)",
  "timeComplexity": "O(n log n)",
  "spaceComplexity": "O(n)",
  "needsRevision": true,
  "dateSolved":  "2026-01-15T10:30:00.000Z"
}
```

ERROR RESPONSES

400	A problem named "<name>" already exists, or A problem with this link already exists
401	Not authorized (ownership check — same message text as a missing/invalid token, per the controller)
404	Problem not found

DELETE `/api/problems/:id` Required

Deletes a problem. Ownership is checked.

SUCCESS RESPONSE 200

```
{
  "message": "Problem removed"
}
```

ERROR RESPONSES

401	Not authorized (ownership check — same message text as a missing/invalid token, per the controller)
404	Problem not found

Dashboard APIs

/api/dashboard

Aggregated analytics that power the Dashboard page.

GET `/api/dashboard` **Required**

Returns topic counts, problem counts, per-topic mastery scores, weak topics, topics/problems flagged for revision, and topics/problems added in the last 7 days. Mastery Score: Easy = 1 point, Medium = 2 points, Hard = 3 points. The score (and the 'solved' count it's based on) is calculated only from problems whose status is 'Solved' OR whose needsRevision flag is true — a problem marked needsRevision counts toward mastery even if its status isn't 'Solved'. A topic is classified as weak when its mastery score is below 7 AND it has at least one problem logged — topics with zero problems are excluded from the weak-topics list entirely (they aren't started yet, not weak). This is the core analytics endpoint and feeds weak-topic data into the AI Study Plan generator.

SUCCESS RESPONSE 200

Note: each topicWiseProblems/weakTopics entry identifies the topic by its name string in a field called "topic" (not an ID, and not split into topicId/topicName). problems.solvedProblems (the top-level total) is derived by summing each topic's solved count from topicWiseProblems, not from a separate database query — this guarantees it stays consistent with the per-topic breakdown.

```
{
  "topics":            {
    "totalTopics": 12,
    "completedTopics":  5,
    "inProgressTopics":  4,
    "pendingTopics":  3
  },
  "problems":               {
    "totalProblems": 84,
    "solvedProblems":  60,
    "easyProblems":  30,
    "mediumProblems":  40,
    "hardProblems":  14
  },
  "topicWiseProblems":  [
    {
      "topic": "Dynamic Programming",
      "count": 10,
      "solved": 4,
      "remaining": 6,
      "easy": 3,
      "medium": 1,
      "hard": 0,
      "lastSolvedDate":  "2026-01-14T10:30:00.000Z",
      "masteryScore":  5
}
],

  "weakTopics":  [
    {
      "topic": "Dynamic Programming",
      "masteryScore": 5
}
],

  "revisionTopics":  [
    {
      "_id":        "66b1f2a4c9d2e1a1b2c3d4e8",
      "name": "Linked List",
      "status": "In Progress"
}
],

  "revisionProblems":  [
    {
      "_id":        "66b1f2a4c9d2e1a1b2c3d4f3",
      "name":    "Merge    K    Sorted    Lists",
      "difficulty": "Hard",
      "link":          "https://leetcode.com/problems/merge-k-sorted-lists/",
      "topic": {
        "_id":      "66b1f2a4c9d2e1a1b2c3d4e7",
        "name": "Graphs"
      }
}
],

  "topicsThisWeek":  2,
  "problemsThisWeek":  9
}
```

ERROR RESPONSES

401	Not authorized

Daily Goal APIs

/api/goals

Daily problem-solving goal tracking, used by the Daily Goal calendar page. Dates are stored as plain strings (YYYY-MM-DD), not a Date type.

GET `/api/goals` **Required**

Returns today's goal. If a goal document already exists for today, achieved is recalculated from problems solved today and saved if it changed, then the goal is returned. If no goal exists yet for today, no document is created — the response returns a computed default of {target: 0, achieved: <today's count>, date: today} without persisting anything. A real Goal document is only created once the user calls POST /goals.

Response above shows the case where a goal already exists. If none exists yet for today, the response is instead {"target": 0, "achieved": 2, "date": "2026-01-15"} — with no _id, since nothing was saved to the database.

SUCCESS RESPONSE 200


```
{
  "_id": "66b1f2a4c9d2e1a1b2c3d4g1",
  "target": 5,
  "achieved": 2,
  "date":  "2026-01-15"
}
```


ERROR RESPONSES

401	Not authorized

POST `/api/goals` Required

Sets (upserts) the target problem count for the given date. Note: the date field in the request body is currently accepted but not actually used by the controller — it always reads/writes against today's date (computed server-side), regardless of what date value is sent. One goal document exists per user per date (unique compound index). achieved is recalculated from today's solved problems at save time and returned alongside target. Note: this route has no validation middleware and the controller doesn't manually check target's presence/type — target is required at the schema level, so a missing/invalid target throws a Mongoose ValidationError, which the generic catch block returns as a 500 (not a clean 400) with the raw Mongoose error message.

REQUEST BODY

```
{
  "target": 5,
  "date": "2026-01-15 (accepted but currently ignored \u2014 see note below)"
}
```

The date field is currently ignored by the server; it always reads/writes against the server-computed today, regardless of what date is sent here. Sending a different date will not create/update a goal for that date.

SUCCESS RESPONSE 200

```
{
  "_id": "66b1f2a4c9d2e1a1b2c3d4g1",
  "target": 5,
  "achieved": 2,
  "date":  "2026-01-15"
}
```

ERROR RESPONSES

401	Not authorized
500	target missing/invalid — surfaces as a raw Mongoose validation error message, not a clean 400

GET /api/goals/history

Required

Returns goal history for a given month/year, matched via regex on the date-string prefix. Powers the Daily Goal calendar view.

QUERY PARAMETERS

month	optional, default: current month
year	optional, default: current year

SUCCESS RESPONSE 200

```
[
  {
    "date": "2026-01-14",
    "target": 3,
    "achieved": 3
  },
  {
    "date": "2026-01-15",
    "target": 5,
    "achieved": 2
  }
]
```

ERROR RESPONSES

401	Not authorized

AI Study Plan APIs

/api/ai

AI-generated personalized DSA study plans via Google Gemini (gemini-flash-latest). Note: Gemini's Flash-tier models have had intermittent 503 'high demand' outages since Feb 2026 (a known Google-side issue). The backend retries automatically — up to 4 total attempts (1 initial attempt + up to 3 retries), with exponential backoff of 1s, 2s, and 4s between attempts — before returning a clean 503 error.

POST `/api/ai/study-plan` Required

Generates a personalized study plan based on the weakTopics array sent in the request body. weakTopics is sourced from the weakTopics field returned by GET /api/dashboard — but that field returns an array of objects ({topic, masteryScore}), while this endpoint expects a plain array of topic name strings. The client must extract the topic field from each object and build a new string array before forwarding it here (e.g. dashboardResponse.weakTopics.map(t => t.topic)). This endpoint does not automatically fetch weak topics itself. If a plan already exists for today and 'regenerate' isn't set, the cached plan is returned with no AI call (fromCache: true). Maximum 2 generations per day (1 initial + 1 regenerate) — generationCount in the response tracks this: 1 = initial generation, 2 = after one regenerate (the daily max).

Rate limit: 2 generations/day (app-level, per user)

REQUEST BODY

```
{
  "weakTopics": [
    "Dynamic Programming",
    "Graphs"
  ],
  "totalDays": 7,
  "regenerate": false
}
```

SUCCESS RESPONSE 200

```
{
  "fromCache": false,
  "date": "2026-01-15",
  "weakTopics": [
    "Dynamic Programming",
    "Graphs"
  ],
  "totalDays": 7,
  "generationCount": 1,
  "completedTasks": [],
  "studyPlan": [
    {
      "day": 1,
      "topic": "Dynamic Programming",
      "concepts": [
        "Memoization",
        "Tabulation"
      ],
      "problems": [
        {
          "name": "Climbing Stairs",
          "difficulty": "Easy",
          "platform": "LeetCode"
        },
        {
          "name": "Longest Increasing Subsequence",
          "difficulty": "Medium",
          "platform": "LeetCode"
        }
      ]
    }
  ],
  "tips": [
    "Revisit recursion basics before starting DP"
  ]
}
```


Response shown above includes only Day 1 of the full 7-day plan (matching totalDays: 7 from the request) for brevity — the real response's studyPlan array has one entry per day, all 7 with the same structure shown for Day 1.

ERROR RESPONSES

400	weakTopics missing
401	Not authorized
429	Daily regeneration limit reached, or Gemini quota exhausted
502	Gemini returned invalid/unparseable JSON
503	Gemini overloaded — retried automatically, please try again shortly

GET `/api/ai/study-plan/today` **Required**

Returns today's study plan if one exists. Returns 200 with {exists: false} if none has been generated yet — not a 404.

SUCCESS RESPONSE 200


```
{
  "exists": true,
  "date": "2026-01-15",
  "weakTopics": [
    "Dynamic Programming"
  ],
  "totalDays": 1,
  "completedTasks": [
    "day1-task1"
  ],
  "studyPlan": [
    {
      "day": 1,
      "topic": "Dynamic Programming",
      "concepts": [
        "Memoization",
        "Tabulation"
      ],
      "problems": [
        {
          "name": "Climbing Stairs",
          "difficulty": "Easy",
          "platform": "LeetCode"
        }
      ]
    }
  ],
  "tips": [
    "Revisit recursion basics before starting DP"
  ]
}
```


ERROR RESPONSES

401	Not authorized

GET `/api/ai/study-plan/history` **Required**

Returns past study plans, most recent first. Only summary fields are returned (date, weakTopics, totalDays, createdAt), not the full plan content.

SUCCESS RESPONSE 200


```
[
  {
    "date": "2026-01-14",
    "weakTopics": [
      "Graphs"
    ],
    "totalDays": 3,
    "createdAt": "2026-01-14T08:15:00.000Z"
  },
  {
    "date": "2026-01-10",
    "weakTopics": [
      "Dynamic Programming",
      "Trees"
    ],
    "totalDays": 5,
    "createdAt": "2026-01-10T09:00:00.000Z"
  }
]
```


ERROR RESPONSES

401	Not authorized

GET	/api/ai/study-plan/date/:date	Required

Returns the full study plan for a specific date (YYYY-MM-DD).

SUCCESS RESPONSE 200


```
{
  "date": "2026-01-14",
  "weakTopics": [
    "Graphs"
  ],
  "totalDays": 1,
  "completedTasks": [],
  "studyPlan": [
    {
      "day": 1,
      "topic": "Graphs",
      "concepts": [
        "BFS",
        "DFS"
      ],
      "problems": [
        {
          "name": "Number of Islands",
          "difficulty": "Medium",
          "platform": "LeetCode"
        }
      ]
    }
  ],
  "tips": [
    "Draw out graph traversals on paper before coding"
  ]
}
```


ERROR RESPONSES

401	Not authorized
404	No plan found for this date

PATCH `/api/ai/study-plan/progress` Required

Marks tasks complete/incomplete for a given day's plan by replacing the completedTasks array (the sent array overwrites the previous state — it is not additive). completedTasks holds task identifier strings in the format day{N}-task{M}, where N is the plan day number and M is the problem's position within that day (e.g. "day1-task2" = the 2nd problem listed under Day 1). These identifiers are generated on the frontend from the plan's day/problem indices — they are not stored as separate IDs in the database.

REQUEST BODY

```
{
  "date": "2026-01-15",
  "completedTasks": [
    "day1-task1",
    "day1-task2"
  ]
}
```

SUCCESS RESPONSE 200

```
{
  "completedTasks": [
    "day1-task1",
    "day1-task2"
  ]
}
```

ERROR RESPONSES

400	date or completedTasks missing
401	Not authorized
404	No plan exists for that date

Help & Support APIs

/api/support

Contact, bug report, and feature request submission. All three send an email notification via the Gmail API (OAuth2, sent through the support inbox's authenticated Google account). name/email on every ticket are taken from the authenticated user (req.user), never from the request body, so submissions can't be spoofed. No field-level validation middleware (e.g. length limits on subject/message) is currently applied to these routes — confirmed by the absence of a validateSupport export in validate.js.

POST `/api/support/contact` — Required

Submits a general contact message.

REQUEST BODY

```
{
  "subject": "Question about study plans",
  "message": "How often can I regenerate my AI study plan?"
}
```

SUCCESS RESPONSE 201

```
{
  "message": "Your message has been sent! We'll get back to you within 24 hours.",
  "ticket": {
    "_id": "66b1f2a4c9d2e1a1b2c3d4h1",
    "type": "contact",
    "name": "Varinda",
    "email": "varinda@example.com",
    "subject": "Question about study plans",
    "message": "How often can I regenerate my AI study plan?",
    "createdAt": "2026-01-15T12:00:00.000Z"
  }
}
```

ERROR RESPONSES

401	Not authorized
500	Email delivery or server error

POST `/api/support/bug` **Required**

Submits a bug report with category and priority.

REQUEST BODY

```
{
  "subject": "Dashboard mastery score not updating",
  "message": "Solved 3 problems but mastery score stayed the same after refresh.",
  "category": "Backend",
  "priority": "Medium"
}
```

category must be one of: UI | Backend | AI | Performance. priority must be one of: Low | Medium | High.

SUCCESS RESPONSE 201

```
{
  "message": "Bug report submitted. Thanks for helping us improve!",
  "ticket": {
    "_id": "66b1f2a4c9d2e1a1b2c3d4h2",
    "type": "bug",
    "name": "Varinda",
    "email": "varinda@example.com",
    "subject": "Dashboard mastery score not updating",
    "message": "Solved 3 problems but mastery score stayed the same after refresh.",
    "category": "Backend",
    "priority": "Medium",
    "createdAt":  "2026-01-15T12:05:00.000Z"
  }
}
```

ERROR RESPONSES

401	Not authorized
500	Email delivery or server error

POST `/api/support/feature` Required

Submits a feature suggestion.

REQUEST BODY

```
{
  "subject": "Dark mode toggle",
  "message": "Would love a dark mode option in the settings."
}
```

SUCCESS RESPONSE 201

```
{
  "message": "Suggestion submitted. We appreciate your feedback!",
  "ticket": {
    "_id": "66b1f2a4c9d2e1a1b2c3d4h3",
    "type": "feature",
    "name": "Varinda",
    "email": "varinda@example.com",
    "subject": "Dark mode toggle",
    "message": "Would love a dark mode option in the settings.",
    "createdAt": "2026-01-15T12:10:00.000Z"
  }
}
```

ERROR RESPONSES

401	Not authorized
500	Email delivery or server error